Day 4

Assignment

1] https://leetcode.com/problems/linked-list-cycle-ii

Code

Java Auto

```java
public class Solution {
    public ListNode detectCycle(ListNode head) {
      ListNode slow=head;
      ListNode fast=head;
      while(fast!=null && fast.next!=null){
        slow=slow.next;
        fast=fast.next.next;
        if(fast==slow){
            break;
        }
      }
      if(fast==null||fast.next==null){
        return null;
      }
      //for finding starting point of cycle
      while(head!=slow)
        {
            head=head.next;
            slow=slow.next;
        }
    return head;
    }
}
```

Saved

Testcase | Test Result

Accepted Runtime: 0 ms

• Case 1 • Case 2 • Case 3

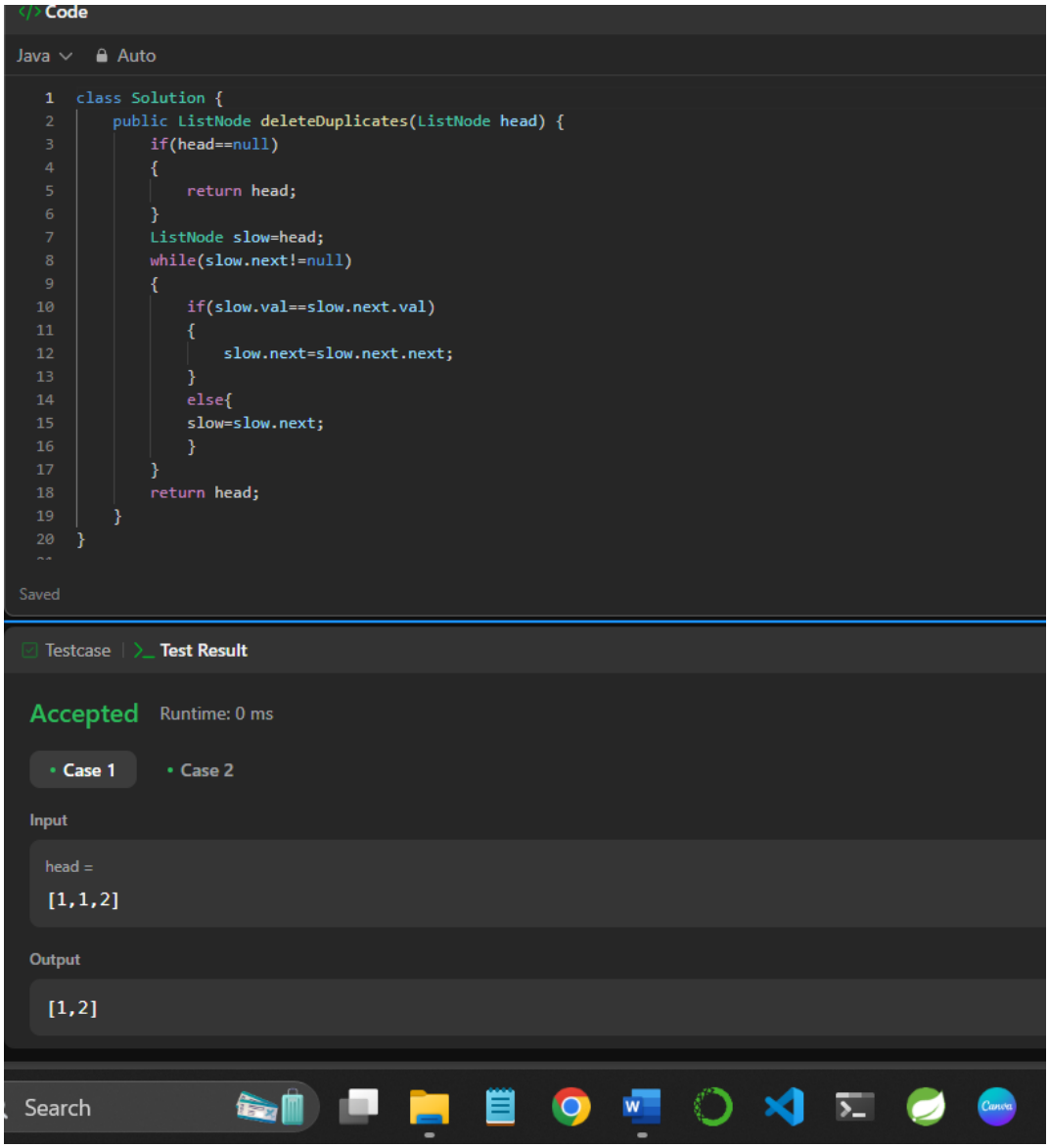
Input

head =

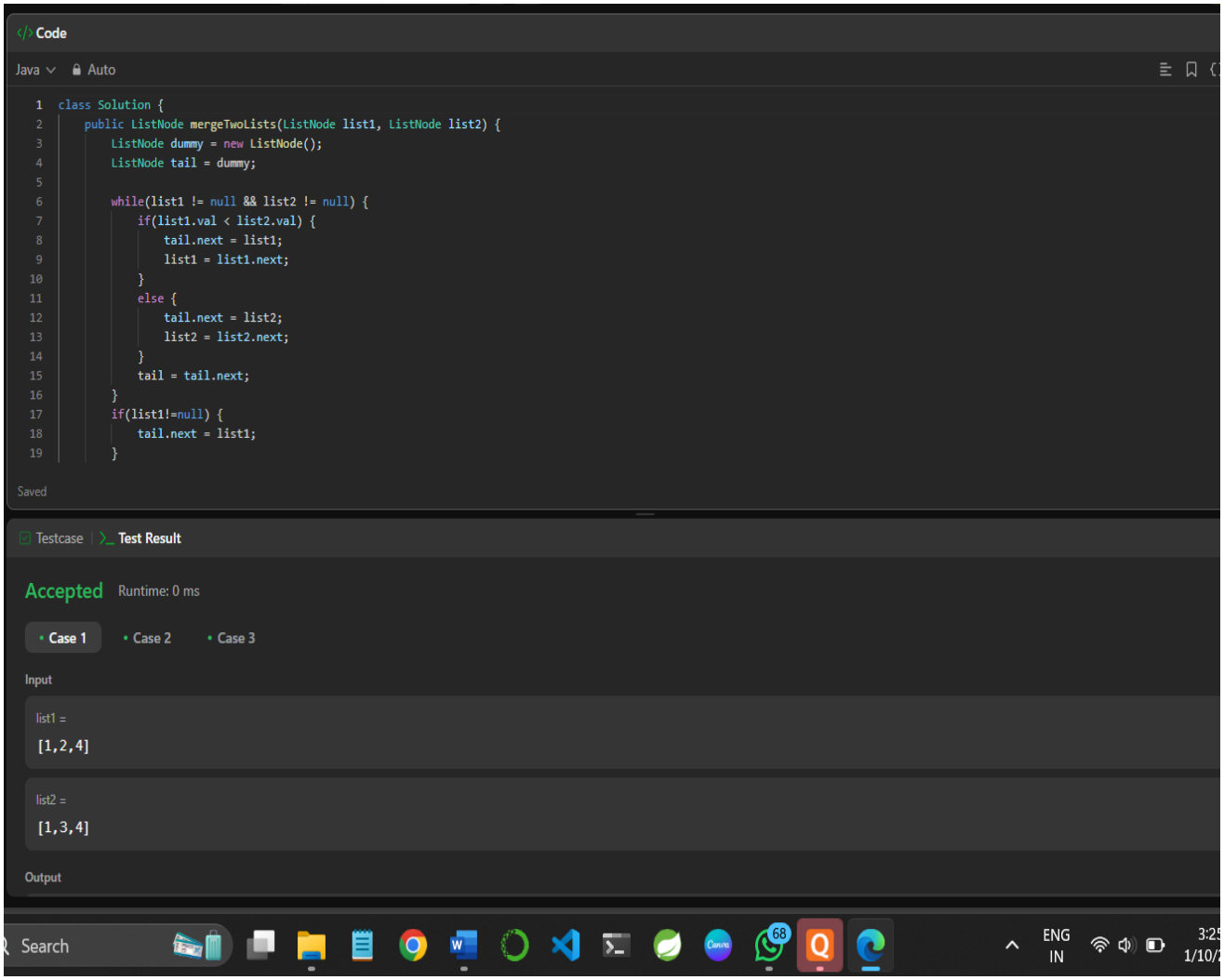
[3,2,0,-4]

Search

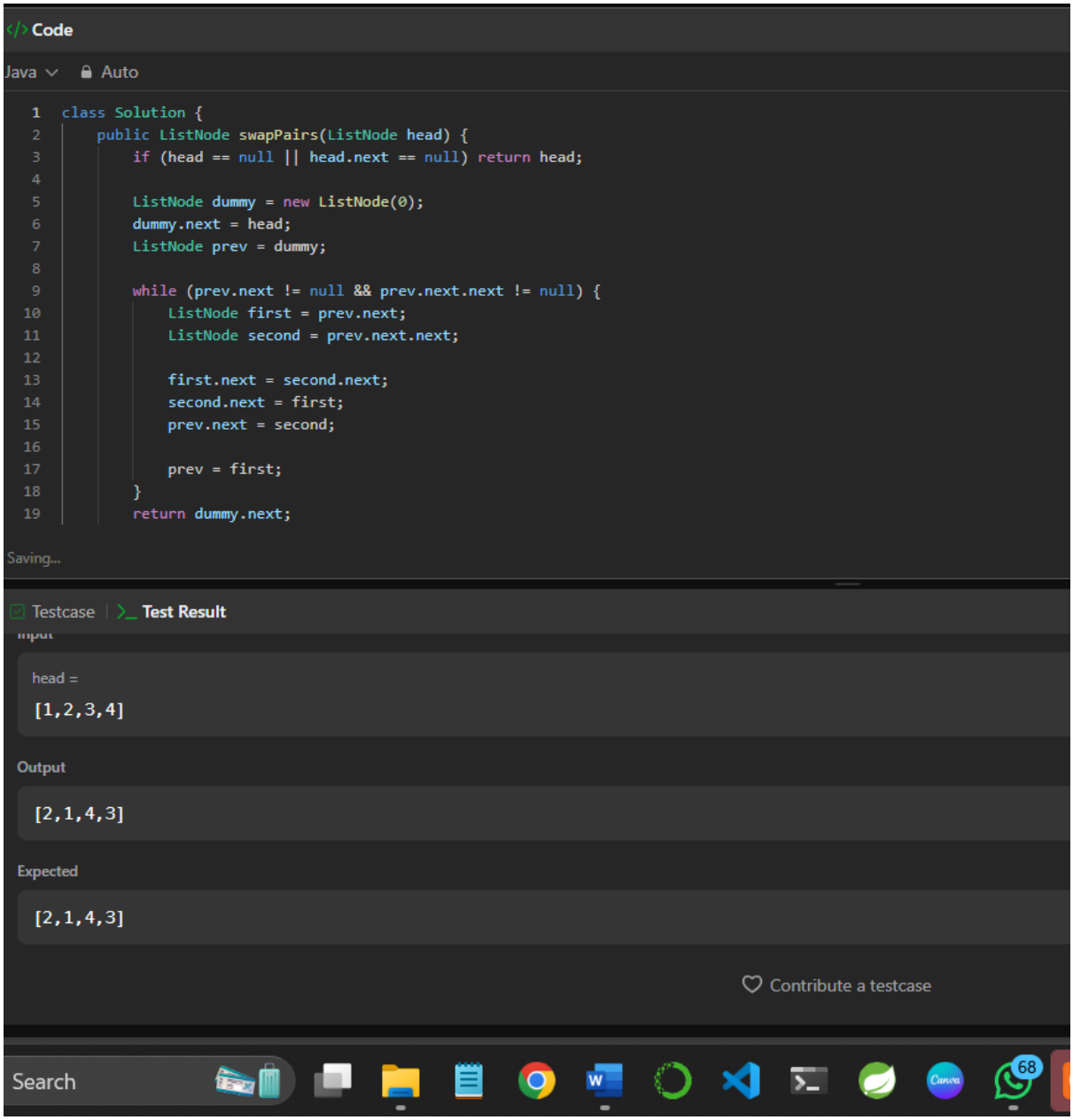
2] https://leetcode.com/problems/remove-duplicates-from-sorted-list



3] https://leetcode.com/problems/merge-two-sorted-lists



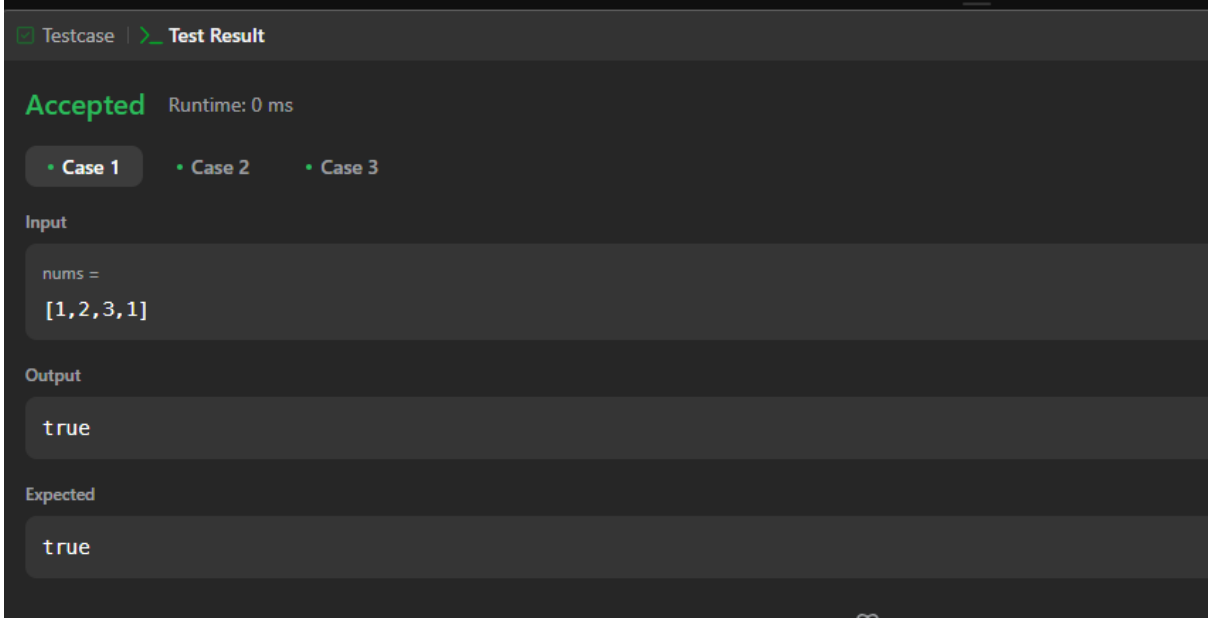
4] https://leetcode.com/problems/swap-nodes-in-pairs



5] https://leetcode.com/problems/contains-duplicate/

```java
class Solution {
    public boolean containsDuplicate(int[] nums) {
        Arrays.sort(nums);
        int n=nums.length;
        for(int i=1;i<n;i++)
        {
            if(nums[i]==nums[i-1]){
            return true;}
        }
        return false;
    }
}
```

Saved



6] https://leetcode.com/problems/longest-palindrome/

Java Auto

```java
class Solution {
    public int longestPalindrome(String s) {
        int oddCount = 0; // Count of characters with odd frequency
        HashMap<Character, Integer> frequencyMap = new HashMap<>();

        for (char ch : s.toCharArray()) {
            frequencyMap.put(ch, frequencyMap.getOrDefault(ch, 0) + 1);
            if (frequencyMap.get(ch) % 2 == 0)
                oddCount--;
            else
                oddCount++;
        }
        if (oddCount > 1)
            return s.length() - oddCount + 1;
        return s.length();
    }
}
```

Saved